

Deploy Model

Ngày 29 tháng 11 năm 2025

1 So Sánh Tính Năng Chính

Hai framework này phục vụ cho việc xây dựng ứng dụng web và API, nhưng khác biệt lớn nhất nằm ở kiến trúc đồng bộ/không đồng bộ và việc tích hợp các công cụ hiện đại.

Bảng 1: So Sánh Tính Năng Cốt Lõi

Tính Năng	FastAPI	Flask
Kiến Trúc	Hiện đại, Không Đồng Bộ (Async) dựa trên Starlette (ASGI).	Cổ điển, Đồng Bộ (Sync) dựa trên Werkzeug (WSGI).
Hiệu Suất	Rất Cao. Tận dụng <code>async/await</code> để xử lý đồng thời (concurrency) hiệu quả cho các tác vụ I/O-bound.	Trung Bình. Cần nhiều worker/process để đạt concurrency.
Xác Thực Dữ Liệu	Tích hợp sẵn Pydantic. Tự động validate input và serialize output bằng Python <i>Type Hints</i> .	Không Tích hợp; yêu cầu sử dụng thư viện bên ngoài (ví dụ: Marshmallow) và cài đặt thủ công.
Tài Liệu API	Tự động tạo tài liệu tương tác chuẩn OpenAPI (<i>Swagger UI, ReDoc</i>).	Không Tự động; cần extension hoặc viết tay.
Cú Pháp	Tận dụng tối đa <i>Python Type Hints</i> để định nghĩa schema và Dependency Injection.	Cú pháp tối giản, dễ hiểu.

2 Phân Tích Chi Tiết Hiệu Suất

Sự khác biệt về hiệu suất chủ yếu đến từ kiến trúc xử lý yêu cầu:

- **FastAPI (ASGI):** Xử lý các yêu cầu I/O (như truy vấn CSDL, gọi API bên ngoài) mà không chặn luồng chính (non-blocking). Điều này cho phép server xử lý hàng ngàn kết nối cùng một lúc một cách hiệu quả.

$$\text{Efficiency} \propto \text{Concurrency (I/O-bound)}$$

- **Flask (WSGI):** Mỗi yêu cầu thường sẽ chặn luồng xử lý (blocking) cho đến khi hoàn tất. Để xử lý nhiều yêu cầu, cần nhân bản các tiến trình/worker, dẫn đến tiêu tốn bộ nhớ và CPU nhiều hơn.

3 Cách Lựa Chọn Framework

3.1 Khi Nào Dùng FastAPI

FastAPI là lựa chọn hàng đầu cho các dự án sau:

- **API Hiệu Suất Cao:** Khi cần xây dựng **Microservices** hoặc **Backend API** đòi hỏi tốc độ xử lý nhanh và khả năng chịu tải lớn (high throughput).
- **Độ Tin Cậy Dữ Liệu Cao:** Khi cần **Type Safety** (an toàn về kiểu dữ liệu) và **Validation** tự động để đảm bảo chất lượng của API.
- **Chuẩn Hóa API:** Khi cần tự động tạo tài liệu **OpenAPI** (Swagger/ReDoc) để thuận tiện cho các nhà phát triển frontend và bên thứ ba.

3.2 Khi Nào Dùng Flask

Flask phù hợp với các trường hợp ưu tiên sự đơn giản và linh hoạt:

- **Dự án Nhỏ và MVP:** Khi cần triển khai một ý tưởng nhanh chóng với codebase tối giản nhất.
- **Web App Truyền Thống:** Khi xây dựng ứng dụng web có sử dụng **Server-Side Rendering** (ví dụ: dùng Jinja2 để render HTML).
- **Học Tập:** Là điểm khởi đầu tuyệt vời cho những người mới làm quen với phát triển web Python nhờ vào sự đơn giản và dễ hiểu.